University of Asia Pacific

Department of CSE



Lab Report-6

Priority Scheduling
Non-Preemptive & Preemptive Implementation

Course Title: Operating System Lab
Course Code: CSE 402

Submitted By

Tashnimul Hasan Shifat
ID: 23101142
Section: C-2
Department of CSE
University of Asia Pacific

Submitted To

Atia Rahman Orthi
Lecturer
Department of Computer Science and Engineering
University of Asia Pacific

Date of Submission: 03/09/2026

1. Problem

Given the following processes with their Arrival Time (AT), Burst Time (BT), and Priority:

Process	Arrival Time (AT)	Burst Time (BT)	Priority
P1	0	3	3
P2	1	4	2
P3	2	6	4
P4	3	4	6
P5	5	2	10

Implement the following CPU Scheduling Algorithms:

- Priority Non-Preemptive Scheduling
- Priority Preemptive Scheduling

Assume that a smaller priority number indicates a higher priority.

For each algorithm, calculate the following for every process:

- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)

Also, calculate:

- Average Turnaround Time (Average TAT)
- Average Waiting Time (Average WT)

Finally, display the results for all processes and compare the Average TAT and Average WT of both Preemptive and Non-Preemptive Priority Scheduling algorithms to determine which algorithm performs better for the given processes.

2. Source Code

```
processes_base = [
    {"pid": "P1", "at": 0, "priority": 3, "bt": 3},
    {"pid": "P2", "at": 1, "priority": 2, "bt": 4},
    {"pid": "P3", "at": 2, "priority": 4, "bt": 6},
    {"pid": "P4", "at": 3, "priority": 6, "bt": 4},
    {"pid": "P5", "at": 5, "priority": 10, "bt": 2},
]


def non_preemptive(processes):
    n = len(processes)
    completed, completed_pids = [], set()
    current_time = 0

    while len(completed) < n:
        ready = [p for p in processes
                 if p["at"] <= current_time and p["pid"] not in
completed_pids]
        if not ready:
            current_time = min(p["at"] for p in processes if p["pid"]
not in completed_pids)
            continue

        current = min(ready, key=lambda p: (p["priority"], p["at"]))
        start = current_time
        finish = start + current["bt"]
        tat = finish - current["at"]
        wt = tat - current["bt"]

        completed.append({**current, "finish": finish, "turnaround":
tat, "waiting": wt})
        completed_pids.add(current["pid"])
        current_time = finish

    return completed
```

```
def preemptive(processes):
    n = len(processes)
    remaining = {p["pid"]: p["bt"] for p in processes}
    finish_time = {}
```

```
    current_time = 0
    completed = 0

    while completed < n:
        ready = [p for p in processes
                 if p["at"] <= current_time and remaining[p["pid"]] >
0]
        if not ready:
            current_time += 1
            continue

        current = min(ready, key=lambda p: (p["priority"], p["at"]))
        pid = current["pid"]
        remaining[pid] -= 1
        current_time += 1

        if remaining[pid] == 0:
            finish_time[pid] = current_time
            completed += 1

    results = []
    for p in processes:
        ct = finish_time[p["pid"]]
        tat = ct - p["at"]
        wt = tat - p["bt"]
        results.append({**p, "finish": ct, "turnaround": tat,
"waiting": wt})
    return results
```

```
def print_table(title, results):
    n = len(results)
    print(f"\n--- {title} ---")

print(f"{'Process':<8}{'AT':<5}{'Priority':<10}{'BT':<5}{'Finish':<8}{
'TAT':<6}{'WT':<5}")




   total_tat = total_wt = 0
    for r in results:

print(f"{r['pid']:<8}{r['at']:<5}{r['priority']:<10}{r['bt']:<5}"
```

```
f"{r['finish']:<8}{r['turnaround']:<6}{r['waiting']:<5}")
        total_tat += r["turnaround"]
        total_wt += r["waiting"]
    avg_tat = total_tat / n
    avg_wt = total_wt / n
    print(f"Average TAT = {avg_tat:.2f} ms | Average WT = {avg_wt:.2f}
ms")
    return avg_tat, avg_wt

np_results = non_preemptive([p.copy() for p in processes_base])
pre_results = preemptive([p.copy() for p in processes_base])

np_avg_tat, np_avg_wt = print_table("Non-Preemptive Priority
Scheduling", np_results)
pre_avg_tat, pre_avg_wt = print_table("Preemptive Priority
Scheduling", pre_results)

print("\n--- Side-by-Side Comparison ---")
print(f"{'Process':<8}{'NP-Finish':<12}{'P-Finish':<10}{'NP-
WT':<8}{'P-WT':<6}")
np_map = {r["pid"]: r for r in np_results}
pre_map = {r["pid"]: r for r in pre_results}
for pid in [p["pid"] for p in processes_base]:
    a, b = np_map[pid], pre_map[pid]

print(f"{pid:<8}{a['finish']:<12}{b['finish']:<10}{a['waiting']:<8}{b[
'waiting']:<6}")

print(f"\n{'Metric':<25}{'Non-Preemptive':<18}{'Preemptive':<12}")
print(f"{'Average Turnaround
(ms)':<25}{np_avg_tat:<18.2f}{pre_avg_tat:<12.2f}")
print(f"{'Average Waiting
(ms)':<25}{np_avg_wt:<18.2f}{pre_avg_wt:<12.2f}")

if pre_avg_wt < np_avg_wt:
    print("\n=> Preemptive scheduling gives lower average waiting time
here.")
elif pre_avg_wt > np_avg_wt:
    print("\n=> Non-Preemptive scheduling gives lower average waiting
time here.")
else:
    print("\n=> Both give the same average waiting time here.")
```

3. Output

```
--- Non-Preemptive Priority Scheduling ---
Process AT   Priority  BT   Finish  TAT   WT
P1      0    3         3    3       3     0
P2      1    2         4    7       6     2
P3      2    4         6    13      11    5
P4      3    6         4    17      14    10
P5      5    10        2    19      14    12
Average TAT = 9.60 ms | Average WT = 5.80 ms

--- Preemptive Priority Scheduling ---
Process AT   Priority  BT   Finish  TAT   WT
P1      0    3         3    7       7     4
P2      1    2         4    5       4     0
P3      2    4         6    13      11    5
P4      3    6         4    17      14    10
P5      5    10        2    19      14    12
Average TAT = 10.00 ms | Average WT = 6.20 ms

--- Side-by-Side Comparison ---
Process NP-Finish   P-Finish  NP-WT   P-WT
P1      3           7         0       4
P2      7           5         2       0
P3      13          13        5       5
P4      17          17        10      10
P5      19          19        12      12

Metric                   Non-Preemptive    Preemptive
Average Turnaround (ms) 9.60              10.00
Average Waiting (ms)     5.80              6.20

=> Non-Preemptive scheduling gives lower average waiting time here.
```

4. Discussion

This experiment implemented and evaluated two variants of Priority CPU Scheduling — Non-Preemptive and Preemptive — using an identical set of five processes with the arrival times, burst times, and priority values specified in the problem statement. A lower priority number was treated as a higher scheduling priority, and the performance of each algorithm was assessed using two standard metrics: Average Turnaround Time (TAT) and Average Waiting Time (WT).

Under **Non-Preemptive Priority Scheduling**, once a process begins execution it is allowed to run to completion without interruption, even if a higher-priority process arrives in the meantime. For the given dataset, this produced an Average TAT of **9.60 ms** and an Average WT of **5.80 ms**.

Under **Preemptive Priority Scheduling**, the currently running process is interrupted whenever a process with a strictly higher priority becomes ready, so the CPU is reassigned immediately. This resulted in an Average TAT of **10.00 ms** and an Average WT of **6.20 ms**. Comparing the two runs, process P1 finishes later under the preemptive scheme (at 7 ms instead of 3 ms) because it is interrupted as soon as the higher-priority process P2 arrives, which raises P1's waiting time from 0 ms to 4 ms. P2, in turn, finishes earlier under preemption (5 ms instead of 7 ms) since it no longer has to wait for P1 to finish. Processes P3, P4, and P5 are unaffected in both cases, as no higher-priority process arrives once they begin executing.

Overall, the non-preemptive algorithm yields a lower Average TAT and a lower Average WT than the Preemptive algorithm for this workload. This happens because the process with the shortest burst time and an early arrival (P1) is allowed to finish quickly without being interrupted, which keeps the overall waiting time low across the process set. In contrast, preempting P1 partway through its execution delays its completion and increases the total waiting time, even though it allows the higher-priority process P2 to start sooner

5. Conclusion

Based on the experimental results, **Priority Non-Preemptive Scheduling is the better-performing algorithm for the given set of processes**, achieving a lower Average Turnaround Time of **9.60 ms** and a lower Average Waiting Time of **5.80 ms**, compared to **10.00 ms** and **6.20 ms** respectively for Priority Preemptive Scheduling.

Non-Preemptive Priority Scheduling therefore performs better for this dataset because none of the processes were interrupted in a way that increased the total waiting time; instead, allowing P1 to run to completion minimized idle switching overhead and kept the overall schedule efficient. However, this advantage is workload-dependent — Preemptive Priority Scheduling remains the more suitable choice in real-time or interactive systems, where a newly arriving high-priority process must gain immediate access to the CPU rather than waiting for a lower-priority process to finish.

6. GitHub: